Without an index this query might take a long time. From psql, run the query again with EXPLAIN (ANALYZE, BUFFERS) in front of it. On PostgreSQL 15 this uses a Parallel Sequential Scan for the temp.users table, which is quite fast but takes more than 500ms and reads over 100,000 buffers. In a web application context where this query was running at a high volume, this time might be unacceptably slow and cause a lot of unnecessary disk access.

You'll fix this by adding an index to the temp.users table. Since you're filtering on the last_name field, add an index to that field.

Since you're adding the index to a non-live system, skip the CONCURRENTLY option – it's not needed and would slow down the creation unnecessarily. To see how long it takes to add, toggle on timing by running the \timing command. With timing enabled, run this statement:

```
CREATE INDEX users_last_name_idx ON temp.users (last_name);
```

After adding the index, analyze the table using the ANALYZE temp.users statement from psql.

Run the query again using EXPLAIN (ANALYZE, VERBOSE). The query now runs in less than 5 milliseconds and there is only one Buffer that's read. The index you just added made this query very efficient with a very low cost and low disk usage. You'll see the name of the index "users_last_name_idx" and a Index Scan as the scan type.

As a Rails developer, you use Active Record Migrations to create indexes, keeping all of the databases the application works with in sync.

Create an Active Record Migration for this Index Creation. This way the index will be created on your local development machine, in any lower environments like CI or staging, and in your production environment.

An example Migration is listed below. This Migration uses the if_not_exists: true option which adds the index only if it doesn't exist. This way you're able to add the Index manually if necessary to production, but backfill a Migration to keep everything in sync. The Migration will run on a live system so the CONCURRENTLY option is used in the conventional way for Active Record.

ruby/add_index_users_last_name.rb

```
class AddIndexUsersLastName < ActiveRecord::Migration[7.0]
  disable_ddl_transaction!

  def change
    add_index :users, :last_name, algorithm: :concurrently,
      if_not_exists: true
  end
end
```